

Ingeniería de Software  
Trabajo Práctico



**Especificación y verificación de un  
sistema de gestión de préstamos de una  
biblioteca**

Gianfranco Polinesi

Abril 2026

# 1 Requerimientos

Se quiere desarrollar un sistema de gestión de libros prestados para una biblioteca. Cada libro es identificado por su ISBN. Se debe llevar un registro de los libros que posee la biblioteca y proveer operaciones para dar de alta o de baja ejemplares de estos libros. Si un libro es prestado, deberá quedar registrado el número del socio a quien fue prestado el libro. El sistema debe estar parametrizado por una cantidad límite de libros que un socio puede retirar. Las funcionalidades que se especificarán serán:

- dar de alta una cierta cantidad de ejemplares de un libro
- marcar un ejemplar del libro como prestado
- consultar cuántos ejemplares de un libro hay disponibles para préstamo

# 2 Especificación

## Designaciones

$i$  es un ISBN  $\approx i \in ISBN$

$s$  es un número de socio  $\approx i \in NROSOCIO$

Cantidad de ejemplares de un libro  $l$  que pertenece a la biblioteca  $\approx libros\ l$

Cantidad prestada de un libro  $l \approx cantPrestada\ l$

Libros prestados al socio  $s \approx prestamos\ s$

Límite de libros que un socio puede retirar a la vez  $\approx limite$

## Tipos básicos y definiciones axiomáticas

$[ISBN, NROSOCIO]$

Se define un tipo para los mensajes de salida de las operaciones:

$MSJ ::= ok \mid noexiste \mid nodisponible \mid excedelimite$

|                       |
|-----------------------|
| $limite : \mathbb{Z}$ |
| $limite > 0$          |

## Variables de estado, estado inicial e invariante

|                                                     |
|-----------------------------------------------------|
| <i>Biblioteca</i>                                   |
| $libros : ISBN \rightarrow \mathbb{N}$              |
| $cantPrestada : ISBN \rightarrow \mathbb{N}$        |
| $prestamos : NROSOCIO \rightarrow \text{seq } ISBN$ |

¿Por qué el codominio de *prestamos* es  $\text{seq } ISBN$  y no  $\mathbb{P } ISBN$ ? Se quiere contemplar los casos en los que un socio retira más de un ejemplar de un mismo libro. Esto no se podría hacer con un conjunto. Se utiliza una secuencia porque admite elementos repetidos.

|                            |
|----------------------------|
| <i>BibliotecaInit</i>      |
| <i>Biblioteca</i>          |
| $libros = \emptyset$       |
| $cantPrestada = \emptyset$ |
| $prestamos = \emptyset$    |

|                                                 |
|-------------------------------------------------|
| <i>BibliotecaInv</i>                            |
| <i>Biblioteca</i>                               |
| $\text{dom } cantPrestada = \text{dom } libros$ |

## Operación de alta

|                                                      |
|------------------------------------------------------|
| <i>AltaLibroNuevo</i>                                |
| $\Delta Biblioteca$                                  |
| $l? : ISBN$                                          |
| $n? : \mathbb{N}$                                    |
| $l? \notin \text{dom } libros$                       |
| $libros' = libros \cup \{l? \mapsto n?\}$            |
| $cantPrestada' = cantPrestada \cup \{l? \mapsto 0\}$ |
| $prestamos' = prestamos$                             |

|                                                           |
|-----------------------------------------------------------|
| <i>AltaLibroExistente</i>                                 |
| $\Delta Biblioteca$                                       |
| $l? : ISBN$                                               |
| $n? : \mathbb{N}$                                         |
| $l? \in \text{dom } libros$                               |
| $libros' = libros \oplus \{l? \mapsto libros \ l? + n?\}$ |
| $cantPrestada' = cantPrestada$                            |
| $prestamos' = prestamos$                                  |

$$AltaLibro \hat{=} AltaLibroNuevo \vee AltaLibroExistente$$

### Operación de consulta

|                                                                                                                                                    |       |
|----------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| $ConsultaDisponibilidadOk$                                                                                                                         | _____ |
| $\exists Biblioteca$<br>$l? : ISBN$<br>$n! : \mathbb{N}$<br>$m! : MSJ$                                                                             |       |
| $l? \in \text{dom } libros$<br>$\text{cantPrestada } l? \leq \text{libros } l?$<br>$n! = \text{libros } l? - \text{cantPrestada } l?$<br>$m! = ok$ |       |
| $LibroNoExiste$                                                                                                                                    | _____ |
| $\exists Biblioteca$<br>$l? : ISBN$<br>$m! : MSJ$                                                                                                  |       |
| $l? \notin \text{dom } libros$<br>$m! = noexiste$                                                                                                  |       |

$$ConsultaDisponibilidad \hat{=} ConsultaDisponibilidadOk \vee LibroNoExiste$$

### Operación de préstamo

Hay dos casos exitosos para la operación *PrestarLibro*: uno cuando el socio retira un libro por primera vez (y no está en el dominio de *prestamos*) y otro en caso contrario.

|                                                                                                                                                                                                                                                                                                                                                           |       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| $PrestarLibroOkSocioNuevo$                                                                                                                                                                                                                                                                                                                                | _____ |
| $\Delta Biblioteca$<br>$l? : ISBN$<br>$s? : NROSOCIO$<br>$m! : MSJ$                                                                                                                                                                                                                                                                                       |       |
| $s? \notin \text{dom } prestamos$<br>$l? \in \text{dom } libros$<br>$\text{cantPrestada } l? < \text{libros } l?$<br>$\text{libros}' = \text{libros}$<br>$\text{cantPrestada}' = \text{cantPrestada} \oplus \{l? \mapsto (\text{cantPrestada } l? + 1)\}$<br>$\text{prestamos}' = \text{prestamos} \oplus \{s? \mapsto \langle l? \rangle\}$<br>$m! = ok$ |       |

|                                                                                      |
|--------------------------------------------------------------------------------------|
| <i>PrestarLibroOkSocioExiste</i>                                                     |
| $\Delta Biblioteca$                                                                  |
| $l? : ISBN$                                                                          |
| $s? : NROSOCIO$                                                                      |
| $m! : MSJ$                                                                           |
| $s? \in \text{dom } prestamos$                                                       |
| $\#(prestamos\ s?) < limite$                                                         |
| $l? \in \text{dom } libros$                                                          |
| $cantPrestada\ l? < libros\ l?$                                                      |
| $libros' = libros$                                                                   |
| $cantPrestada' = cantPrestada \oplus \{l? \mapsto (cantPrestada\ l? + 1)\}$          |
| $prestamos' = prestamos \oplus \{s? \mapsto prestamos\ s? \cap \langle l? \rangle\}$ |
| $m! = ok$                                                                            |

$$PrestarLibroOk \hat{=} PrestarLibroOkSocioNuevo \vee PrestarLibroOkSocioExiste$$

|                                 |
|---------------------------------|
| <i>SocioExcedeLimite</i>        |
| $\Xi Biblioteca$                |
| $s? : NROSOCIO$                 |
| $m! : MSJ$                      |
| $s? \in \text{dom } prestamos$  |
| $\#(prestamos\ s?) \geq limite$ |
| $m! = excedelimite$             |

|                                 |
|---------------------------------|
| <i>LibroNoDisponible</i>        |
| $\Xi Biblioteca$                |
| $l? : ISBN$                     |
| $m! : MSJ$                      |
| $l? \in \text{dom } libros$     |
| $cantPrestada\ l? = libros\ l?$ |
| $m! = nodisponible$             |

$$PrestarLibro \hat{=} PrestarLibroOk \vee SocioExcedeLimite \vee LibroNoDisponible \vee LibroNoExiste$$

### 3 Traducción a {log}

La traducción de la especificación se encuentra en el archivo `biblioteca.slog`. Como se utilizan secuencias, es necesario cargar la librería `osets.slog` antes de consultar el archivo.

Al descargar las condiciones de verificación, se crea el archivo `biblioteca-vc.slog`. Para poder consultarlo exitosamente, fue necesario editarlo y agregarle la línea

```
:- add_lib("osets.slog").
```

## 4 Simulaciones

Al existir un parámetro en la especificación, antes de poder realizar alguna simulación, se debe utilizar el comando:

```
setpp(limite_1).
```

### 4.1 Entorno NEXT

#### Simulación 1

Agregar un libro nuevo, luego un libro existente

```
[initial]:[invTipo, bibliotecaInv]
>> altaLibro(L:"9788420471839", N:[[2, 1]])
>> consultarLibro(L:"9788420471839").
```

Execution trace is:

```
Libros = {},
CantPrestada = {},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:2)
Libros = {[9788420471839,2]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:1)
Libros = {[9788420471839,2],[9788420471839,1]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
invTipo check failed
----> consultarLibro(L:"9788420471839") failed, execution
aborted
```

Se observa que se rompe el invariante de que `Libros` es función (el libro aparece dos veces en el dominio)

Al revisar la definición de `altaLibroNuevo`, se descubrió que faltó establecer `DomLibros` como el dominio de `Libros`:

```
altaLibroNuevo(Libros, CantPrestada, Prestamos, L, N, Libros_,
  CantPrestada_, Prestamos_) :-
  dec(DomLibros, set(isbn)) & L nin DomLibros
```

```

& N >= 0
& un(Libros, {[L, N]}, Libros_)
& un(CantPrestada, {[L, 0]}, CantPrestada_)
& Prestamos_ = Prestamos.

```

Se agregó dom(Libros, DomLibros) y se volvió a correr la simulación:

```

Execution trace is:
Libros = {},
CantPrestada = {},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:2)
Libros = {[9788420471839,2]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:1)
Libros = {[9788420471839,1]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
----> consultarLibro(L:"9788420471839")
Libros = {[9788420471839,1]},
CantPrestada = {[9788420471839,0]},
Prestamos = {},
N = 1,
M = ok

```

El resultado es  $N = 1$ , pero se esperaba  $N = 3$ . Esto reveló un error en la definición de `altaLibroExistente`. Faltaba sumar la cantidad de libros existentes. Luego de corregir ese error, el resultado de la simulación es:

```

Execution trace is:
Libros = {},
CantPrestada = {},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:2)
Libros = {[9788420471839,2]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:1)
Libros = {[9788420471839,3]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
----> consultarLibro(L:"9788420471839")
Libros = {[9788420471839,3]},
CantPrestada = {[9788420471839,0]},
Prestamos = {},
N = 3,
M = ok

```

Como se esperaba.

## Simulación 2

Simulaciones que involucran `prestarLibroOk` y `socioExcedeLimite`

```
[initial]:[invTipo, bibliotecaInv]
>> altaLibro(L:["9788420471839", "9788432320767"], N:[2, 1])
>> prestarLibro(L:["9788420471839", "9788432320767"],
  S:"P-4540/3").
```

Parameters:

Limite = 1

Execution trace is:

```
Libros = {},
CantPrestada = {},
Prestamos = {}
----> altaLibro(L:"9788420471839",N:2)
Libros = {[9788420471839,2]},
CantPrestada = {[9788420471839,0]},
Prestamos = {}
----> altaLibro(L:"9788432320767",N:1)
Libros = {[9788420471839,2],[9788432320767,1]},
CantPrestada = {[9788420471839,0],[9788432320767,0]},
Prestamos = {}
----> prestarLibro(L:"9788420471839",S:"P-4540/3")
M = ok,
Libros = {[9788420471839,2],[9788432320767,1]},
CantPrestada = {[9788432320767,0],[9788420471839,1]},
Prestamos = {[P-4540/3,[1,9788420471839]]}
----> prestarLibro(L:"9788432320767",S:"P-4540/3")
M = excede_limite,
Libros = {[9788420471839,2],[9788432320767,1]},
CantPrestada = {[9788432320767,0],[9788420471839,1]},
Prestamos = {[P-4540/3,[1,9788420471839]]}
```

Notar que el resultado es `excede_limite`, como se esperaba. Si se cambia el parámetro límite con:

```
setpp(limite_2).
```

y se vuelve a correr la misma simulación, el estado final queda:

```
M = ok,
Libros = {[9788420471839,2],[9788432320767,1]},
CantPrestada = {[9788420471839,1],[9788432320767,1]},
Prestamos = {[P-4540/3,[2,9788432320767],[1,9788420471839]]}
```



## 4.2 Simulaciones simbólicas

Se activa `groundsol` para realizar algunas simulaciones simbólicas.

### Simulación 3

Se quiere ver un estado donde `prestarLibro` devuelva “no disponible”.

```
dec([Libros, Libros_], libros)
& dec([CantPrestada, CantPrestada_], cantPrestada)
& dec([Prestamos, Prestamos_], prestamos)
& dec(L, isbn) & dec(S, nrosocio)
& prestarLibro(Limite, Libros, CantPrestada, Prestamos, libro, S,
  no_disponible, Libros_, CantPrestada_, Prestamos_).

Libros = {[libro,n3]},
Libros_ = {[libro,n3]},
CantPrestada = {[libro,n3]},
CantPrestada_ = {[libro,n3]},
Prestamos = n2,
Prestamos_ = n2,
S = n1,
Limite = n0
```

Notar que hay `n3` ejemplares de `libro` prestados, que son todos los que tiene la biblioteca. Por lo tanto, en este escenario no quedan libros disponibles para prestar.

### Simulación 4

Se quiere ver en qué ocasión `consultarLibro` da “no existe”.

```
dec([Libros, Libros_], libros)
& dec([CantPrestada, CantPrestada_], cantPrestada)
& dec(L, isbn) & dec(N, int)
& consultarLibro(Libros, CantPrestada, libro, N, no_existe).

Libros = {},
CantPrestada = n1,
N = n0
```

El caso obtenido es cuando la biblioteca no tiene ningún libro.

## 5 Condiciones de verificación

Al chequear las condiciones de verificación se obtuvo:

```
{log}=> check_vcs_biblioteca.
```

```
Checking axioms_sat ... OK
Checking bibliotecaInit_sat_invTipo ... OK
Checking bibliotecaInit_sat_bibliotecaInv ... OK
Checking altaLibro_is_sat ... OK
Checking consultarLibro_is_sat ... OK
Checking prestarLibro_is_sat ... OK
Checking altaLibro_pi_invTipo ... ERROR
Checking altaLibro_pi_bibliotecaInv ... OK
Checking consultarLibro_pi_invTipo ... OK
Checking consultarLibro_pi_bibliotecaInv ... OK
Checking prestarLibro_pi_invTipo ... OK
Checking prestarLibro_pi_bibliotecaInv ... OK
```

```
Total VCs: 12 (discharged: 11, failed: 1, timeout: 0)
Execution time (excluding timeouts): 3.200922727584839 s
yes
```

Como `altaLibro_pi_invTipo` da `ERROR`, se busca un contraejemplo con `vcge`:

```
{log}=> vcgce(altaLibro_pi_invTipo).
```

```
Libros = {},
CantPrestada = {[n1,n0]},
Prestamos = {},
L = n1,
N = 0,
Libros_ = {[n1,0]},
CantPrestada_ = {[n1,0],[n1,n0]},
Prestamos_ = {}
```

El invariante `bibliotecaInv` no se cumple. La operación se aplica a un estado donde `n1` está en el dominio de `CantPrestada` pero no en el dominio de `Libros`.

Para solucionarlo, se agregó este invariante como hipótesis en el predicado que fallaba:

```
altaLibro_pi_invTipo(Libros,CantPrestada,Prestamos,Libros,
  CantPrestada,Prestamos,L,N,Libros_,CantPrestada_,
  Prestamos_) :-
  bibliotecaInv(CantPrestada, Libros) &
  neg(
    invTipo(Libros,CantPrestada,Prestamos) &
    altaLibro(Libros,CantPrestada,Prestamos,L,N,Libros_,
      CantPrestada_,Prestamos_) implies
```

```

        invTipo(Libros_,CantPrestada_,Prestamos_)
    ).

```

Al volver a chequear, se verifica exitosamente:

```

{log}=> consult("biblioteca-vc.slog").
{log}=> check_vcs_biblioteca.

```

```

Checking altaLibro_pi_invTipo ... OK

```

```

Total VCs: 1 (discharged: 1, failed: 0, timeout: 0)
Execution time (excluding timeouts): 0.06345748901367188 s
yes

```

### Nota

En una ocasión la verificación de uno de los lemas me dió TIMEOUT:

```

Checking prestarLibro_pi_bibliotecaInv ... TIMEOUT

```

```

Total VCs: 12 (discharged: 11, failed: 0, timeout: 1)
Execution time (excluding timeouts): 16.007385969161987 s

```

Como `prestarLibro` contiene varias operaciones `oplus` en su definición, probé con el comando:

```

check_vcs_biblioteca(60000, [oplus_fe]).

```

Y `{log}` lo pudo verificar correctamente:

```

Checking prestarLibro_pi_bibliotecaInv ... OK

```

```

Total VCs: 1 (discharged: 1, failed: 0, timeout: 0)
Execution time (excluding timeouts): 0.579350471496582 s

```

## 6 Casos de prueba

### Operación `altaLibro`

Se utiliza la táctica DNF para particionar el árbol en clases de prueba según las precondiciones de la operación. La clase `altaLibro_dnf_1` corresponde al predicado `altaLibroNuevo`, mientras que la clase `altaLibro_dnf_2` corresponde al predicado `altaLibroExistente`. Estas operaciones utilizan los operadores `union` y `oplus`, respectivamente, por lo cual se procede a aplicar la táctica de particiones estándar.

Para una de las clases generadas, se utiliza la táctica II para generar casos de prueba donde N sea mayor que 0.

```

ttf(biblioteca).
applydnf(altaLibro(L,N)).
applysp(altaLibro_dnf_1, un(Libros, {[L, N]}, Libros_)).
applysp(altaLibro_dnf_2, oplus(Libros, {[L, Cant2]}, Libros_)).
applyii(altaLibro_sp_25, N, [1]).
prunett.
gentc.
exportttt.

```

Los casos de prueba generados están en el archivo `biblioteca_altaLibro-tt.slog`

### Operación consultarLibro

En este caso se aplica DNF y luego SC sobre todo el árbol. La táctica SC aquí es interesante porque genera casos donde `Libros`  $\neq \{\}$

```

ttf(biblioteca).
applydnf(consultarLibro(L)).
applysc(consultarLibro_vis, Libros).
prunett.
gentc.
exportttt.

```

Los casos de prueba generados están en el archivo `biblioteca_consultarLibro-tt.slog`

### Operación prestarLibro

Luego de aplicar DNF, se aplica la táctica de particiones estándar sobre las primeras dos clases de prueba utilizando los predicados que aparecen en sus respectivas operaciones.

Para la tercera clase de prueba, que corresponde con el predicado `socioExcedeLimite`, se usa la táctica II para generar casos donde `Limite` asuma diferentes valores.

```

ttf(biblioteca).
applydnf(prestarLibro(L, S)).
applysp(prestarLibro_dnf_1,
        oplus(CantPrestada, {[L, Cplus1]}, CantPrestada_)).
applysp(prestarLibro_dnf_2,
        oplus(Prestamos, {[S, Prestados_]}, Prestamos_)).
applyii(prestarLibro_dnf_3, Limite, [1]).
prunett.
gentc.
exportttt.

```

Los casos de prueba generados están en el archivo `biblioteca_prestarLibro-tt.slog`